

Predicting Academic Outcomes

Using Gaming and AI Student Life Data

CSCI 164 Final Project

- Dataset 1: Gaming Academic Performance → predict grades
- Dataset 2: AI Student Life → predict GPA change
- Main focus: compare models, tune results, and explain why one dataset worked better

Main takeaway:

A good model still depends on having useful data and a predictable target.

Datasets and Targets

Two datasets, two regression targets

Dataset 1: Gaming

- File: Gaming_Academic_Performance.csv
- Rows: 8000
- Target: grades
- Features: gaming hours, study hours, sleep, attendance, stress, addiction score

Dataset 2: AI Student Life

- File: AI_Impact_Student_Life_2026.csv
- Rows: 1500
- Target: GPA_Change
- $\text{GPA_Change} = \text{GPA_Post_AI} - \text{GPA_Baseline}$
- Features: AI tool use, task frequency, time saved, ethics concern, confidence

Code: Load Data and Define Targets

These are the exact target variables used

Dataset 1 target setup

```
df1 = pd.read_csv("Gaming_Academic_Performance.csv")
df1 = df1.drop(columns=["student_id"])

X1 = df1.drop(columns=["grades"])
y1 = df1["grades"]
```

Dataset 2 target setup + leakage removal

```
df2 = pd.read_csv("AI_Impact_Student_Life_2026.csv")
df2["GPA_Change"] = df2["GPA_Post_AI"] - df2["GPA_Baseline"]

X2 = df2.drop(columns=[
    "Student_ID", "GPA_Change",
    "GPA_Post_AI", "GPA_Baseline"
])
y2 = df2["GPA_Change"]
```

Why this matters:

For Dataset 2, GPA_Post_AI and GPA_Baseline were removed after creating GPA_Change to avoid leakage.

Code: Preprocessing Pipeline

Same process used for both datasets

Preprocessing pipeline

```
numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="mean")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(transformers=[
    ("num", numeric_transformer, numeric_features),
    ("cat", categorical_transformer, categorical_features)
])
```

What this does

- Fills missing values if needed
- Scales numeric features
- One-hot encodes categorical features
- Keeps preprocessing inside the model pipeline

Code: Baseline Model Testing

Multiple models were compared before tuning

Baseline model loop

```
models_1 = {
    "Linear Regression": LinearRegression(),
    "KNN Regressor": KNeighborsRegressor(n_neighbors=5),
    "Random Forest Regressor": RandomForestRegressor(random_state=42)
}

for name, model in models_1.items():
    pipeline = Pipeline(steps=[
        ("preprocessor", preprocessor_1),
        ("model", model)
    ])

    pipeline.fit(X1_train, y1_train)
    y_pred = pipeline.predict(X1_test)
```

Why include this?

- Shows 2-3 models were tested
- Uses scikit-learn pipelines
- Creates a fair model comparison
- Matches the Section 2 requirement

Baseline Model Results

Actual results from Section 2

Dataset 1: Gaming Academic Performance

Model	MAE	RMSE	R ²
Random Forest	4.8851	6.3070	0.9206
Linear Regression	5.5039	6.9655	0.9032
KNN Regressor	6.9038	8.6981	0.8491

Dataset 2: AI Student Life

Model	MAE	RMSE	R ²
Linear Regression	0.1189	0.1386	-0.0251
Random Forest	0.1195	0.1403	-0.0517
Decision Tree	0.1623	0.1977	-1.0869

Baseline takeaway

- Dataset 1 already performed well, with Random Forest getting $R^2 = 0.9206$.
- Dataset 2 was much harder to predict, with all baseline R^2 values below 0.

Code: Hyperparameter Tuning

GridSearchCV was used to tune Random Forest

GridSearchCV tuning code

```
param_grid = {
    "model__n_estimators": [50, 100, 200],
    "model__max_depth": [None, 5, 10, 20],
    "model__min_samples_split": [2, 5]
}

grid_search = GridSearchCV(
    rf_pipeline,
    param_grid,
    cv=5,
    scoring="r2",
    n_jobs=-1
)

grid_search.fit(X_train, y_train)
```

Parameters tested

- n_estimators: number of trees
- max_depth: how deep trees can grow
- min_samples_split: minimum split size
- cv = 5 means 5-fold cross-validation

Tuned Random Forest Results

Actual best parameters from Section 3

Dataset	Trees	Depth	Min Split	Test R ²
Gaming RF	200	None	5	0.9215
AI RF	200	5	2	-0.0196

Dataset 1

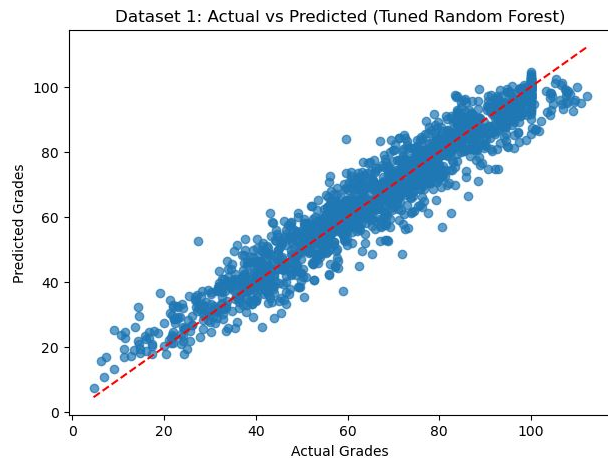
- Tuning slightly improved Random Forest
- Original $R^2 = 0.9206$
- Tuned $R^2 = 0.9215$

Dataset 2

- Tuning improved the Random Forest a little
- Original RF $R^2 = -0.0517$
- Tuned RF $R^2 = -0.0196$

Final Results: Dataset 1

Gaming Academic Performance — Tuned Random Forest



MAE

4.8582

RMSE

6.2739

R^2

0.9215

MSE

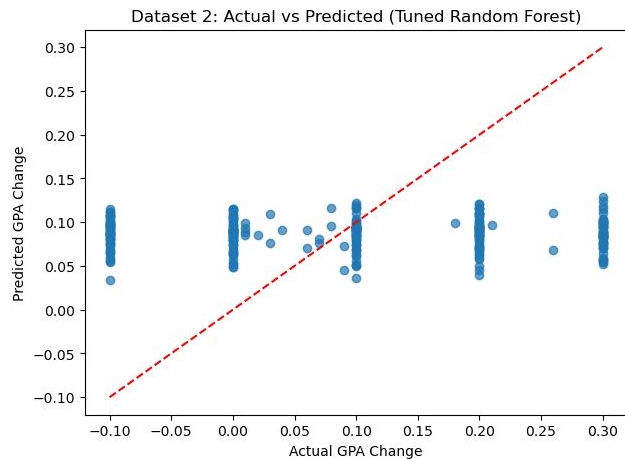
39.3620

Result

The model explained about 92% of the variance in grades, so this dataset had strong predictive structure.

Final Results: Dataset 2

AI Student Life — Tuned Random Forest



MAE

0.1181

RMSE

0.1382

R^2

-0.0196

MSE

0.0191

Result

The model did not outperform a mean-baseline predictor, meaning GPA_Change was hard to predict from the available features.

Literature Comparison

Using the prior works from Section 1

Mahmud et al. (2023)

- Studied gaming, GPA, sleep, and physical activity
- Found long gaming time was linked with lower GPA and less study time
- My Dataset 1 result supports this because gaming/lifestyle features predicted grades well

Wecks et al. (2024)

- Studied generative AI usage and exam performance
- AI users scored 6.71 points lower on average
- My Dataset 2 was related, but GPA_Change was not strongly predictable

Conclusion

What the project showed

- Dataset 1 worked very well because the features had a strong relationship with grades.
- Dataset 2 did not work well because GPA_Change was noisy and probably needed more useful features.
- The biggest lesson is that model performance depends on the data, not just the algorithm.
- Even weak results are useful because they show the limits of the dataset and prediction task.